Lab 1: Java Fundamentals

Algorithms and Data Structures (010153523) | Semester 1/2026

Duration: 2 hours (in-lab) | **Topic:** Classes, objects, OOP basics, `Scanner` I/O

Objectives

- Understand the structure of a Java program: class, `main` method, packages.
- Declare variables of primitive and reference types; use `System.out.println`.
- Read user input using the `Scanner` class.
- Define a class with fields, a constructor, and instance methods (encapsulation).
- Apply the four OOP pillars: encapsulation, inheritance, polymorphism, abstraction.

Quick Reference

Minimal Java program.

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```

Reading input.

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
int n    = sc.nextInt();
double d = sc.nextDouble();
String s = sc.next();       // one token (no spaces)
sc.close();
```

Primitive types. `int`, `long`, `double`, `boolean`, `char`.
String. Immutable reference type; use `+` for concatenation, `.length()`, `.charAt()`.
Access modifiers. `public` (anywhere), `private` (class only), `protected` (class + subclasses).
Workflow. Write `.java` → compile with `javac` → run with `java ClassName`.

Quick Experiments (Try It)

Try It 1: Integer vs. floating-point division

Run this program and record the output:

```
public class DivTest {
    public static void main(String[] args) {
        System.out.println(7 / 2);
        System.out.println(7 / 2.0);
        System.out.println((double) 7 / 2);
    }
}
```

1. What is the difference between the three results?
2. What does the cast `(double)` do?
3. What would happen if you used `int result = 7 / 2.0;`? Try it.

Try It 2: String reference vs. value equality

```java
public class StrEq {
    public static void main(String[] args) {
        String a = new String("hello");
        String b = new String("hello");
        System.out.println(a == b);
        System.out.println(a.equals(b));
    }
}
```

1. Why does `==` print `false` even though the contents are identical?
2. Which method should always be used to compare `String` contents?

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Hello + Your Info *(15 min)*

Write a program that prints three lines: (1) `Welcome to ADS!`, (2) your full name, (3) your student ID. **Goal:** verify the IDE/JDK toolchain works end-to-end.

In-Lab Exercise 2: GPA Calculator *(30 min)*

Read four course grades (as `double`) from the user and compute and print the average GPA formatted to two decimal places using `String.format("%.2f", gpa)`. Label the output clearly, e.g. `Your GPA: 3.56`.

In-Lab Exercise 3: Defining a `Student` Class *(40 min)*

Create a class `Student` with:

- `private` fields: `String name`, `int id`, `double gpa`.
- A constructor that initialises all three fields via `this`.
- Public getters for each field.
- An `@Override` of `toString()` that returns `Student[id=1001, name=Alice, gpa=3.80]`.
- A method `getLetterGrade()` returning `"A"` (≥ 3.5), `"B"` (≥ 3.0), `"C"` (≥ 2.5), or `"F"`.

In `main`, create two `Student` objects, print them, and print each student's letter grade.

In-Lab Exercise 4: Inheritance — `GradStudent` *(35 min)*

Extend `Student` with a subclass `GradStudent` that adds:

- A field `String researchTopic`.
- A constructor calling `super(...)` for the inherited fields.
- An `@Override` of `toString()` that appends `, topic=<value>` to the parent's result.

Demonstrate polymorphism: store both a `Student` and a `GradStudent` in a `Student` reference and call `toString()` on each.

AI Collaboration

Suggested prompts:

- "Explain the difference between `==` and `.equals()` for Java Strings."
- "What does `this` refer to inside a Java constructor?"

- "Why does Java require `super(...)` as the first statement in a subclass constructor?"

Verify: Ask the AI to write a class with a private field accessed directly from outside the class. Confirm the compiler rejects it. Note what error message appears.

Take-Home (Paper Work). Solve the following on paper without using a computer or IDE. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to trace and reason about code during the written exam.

Trace & Predict 1: Trace `toString` output

What does the following `main` print?

```
public class Animal {
    String name;
    Animal(String n) { name = n; }
    public String toString() { return "Animal(" + name + ")"; }
}
public class Dog extends Animal {
    Dog(String n) { super(n); }
    public String toString() { return "Dog(" + name + ")"; }
}
// In main:
Animal a = new Dog("Rex");
System.out.println(a);
```

Find the Bug 1: Fix the constructor

Find and fix all errors:

```
public class Point {
    private int x, y;
    public Point(int x, int y) {
        x = x;   // bug
        y = y;   // bug
    }
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}
```

Write Code on Paper 1: Rectangle class

Write a class `Rectangle` with private fields `width` and `height` (`double`), a constructor, an `area()` method, and a `perimeter()` method. Write a short `main` that creates one `Rectangle` and prints both values.

Submission. Save in-lab source files as `lab1_ex1.java`, ... and submit on the course site. Hand in paper work at the start of the next lab.